DEBASMITA BAGCHI (UG/02/BTCSEAIML/2023/065)

MIDTERM ML LAB EXAM

1.Write a python program to implement Simple Linear Regression for iris using sklearn and plot the confusion matrix for titanic datasets.

Ans>

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

[2]:
```
iris = pd.read_csv("IRIS.csv")

X = iris[['sepal_length']]
y = iris['petal_length']
```

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

[4]:
```
model = LinearRegression()
```

[5]:
```
model.fit(X_train, y_train)
```

[5]:
```
LinearRegression
▶ Parameters
```

[6]:
```
y_pred = model.predict(X_test)
```

[7]:
```
print("Slope (Coefficient):", model.coef_)
print("Intercept:", model.intercept_)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

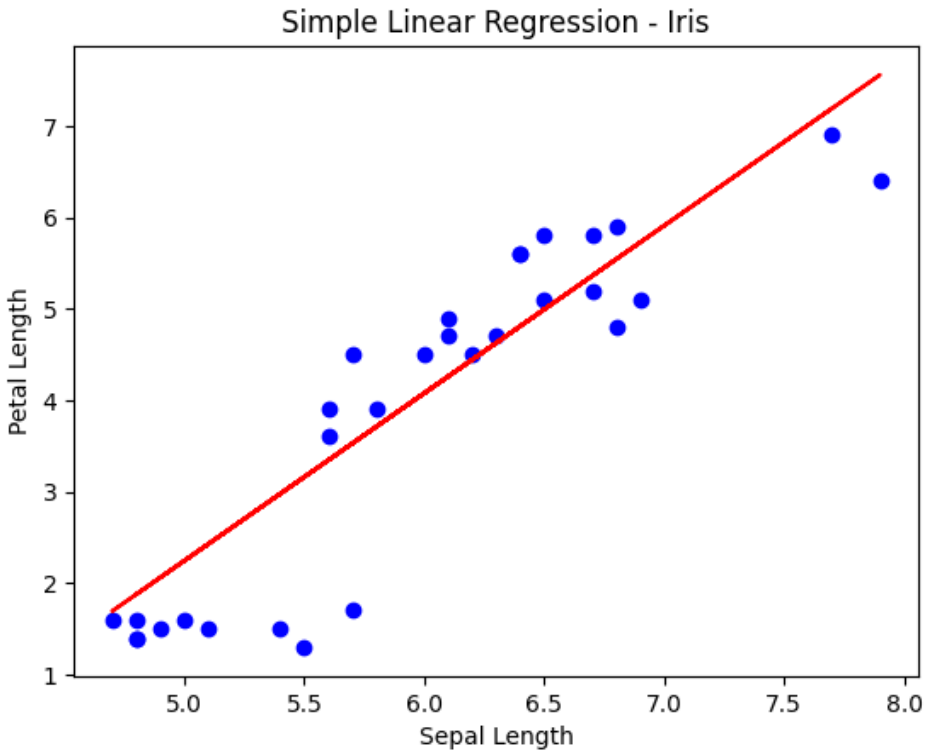
```
Slope (Coefficient): [1.83283683]
Intercept: -6.919754643134638
MSE: 0.5965145177566363
R2 Score: 0.8179909257095002
```

[6]:
```
y_pred = model.predict(X_test)
```

[7]:
```
print("Slope (Coefficient):", model.coef_)
print("Intercept:", model.intercept_)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
Slope (Coefficient): [1.83283683]
Intercept: -6.919754643134638
MSE: 0.5965145177566363
R2 Score: 0.8179909257095002
```

[8]:
```
plt.scatter(X_test, y_test, color='blue')
plt.plot(X_test, y_pred, color='red')
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.title("Simple Linear Regression - Iris")
plt.show()
```



[6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score, classification_report
```

[8]:
```
titanic = pd.read_csv("titanic.csv")
```

[9]:
```
titanic = titanic[['Pclass', 'Sex', 'Age', 'Fare', 'Survived']]
```

[10]:
```
titanic['Sex'] = titanic['Sex'].map({'male': 0, 'female': 1})
```

[11]:
```
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].mean())
```

[12]:
```
X = titanic[['Pclass', 'Sex', 'Age', 'Fare']]
y = titanic['Survived']
```

[13]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[14]:
```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

[14]:
```
▾ LogisticRegression
▶ Parameters
```

[15]:
```
y_pred = model.predict(X_test)
```

[16]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.7988826815642458
```

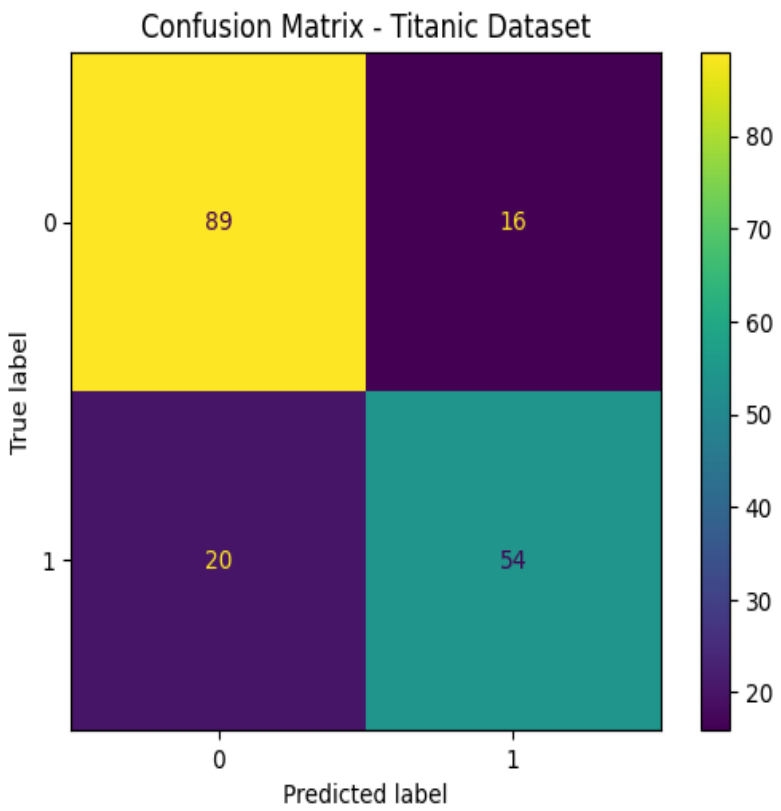
```
Accuracy: 0.7988826815642458

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.85      0.83       105
           1       0.77      0.73      0.75        74

    accuracy                           0.80       179
   macro avg       0.79      0.79      0.79       179
weighted avg       0.80      0.80      0.80       179
```

[17]:
```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Titanic Dataset")
plt.show()
```



2. Write a Python code plot to find out the central tendency and dispersion of data. Verify your output with Scatter Plot.

Ans>

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

[2]:
```
data = pd.read_csv("Heart.csv")
```

[3]:
```
column = data["Age"]
```

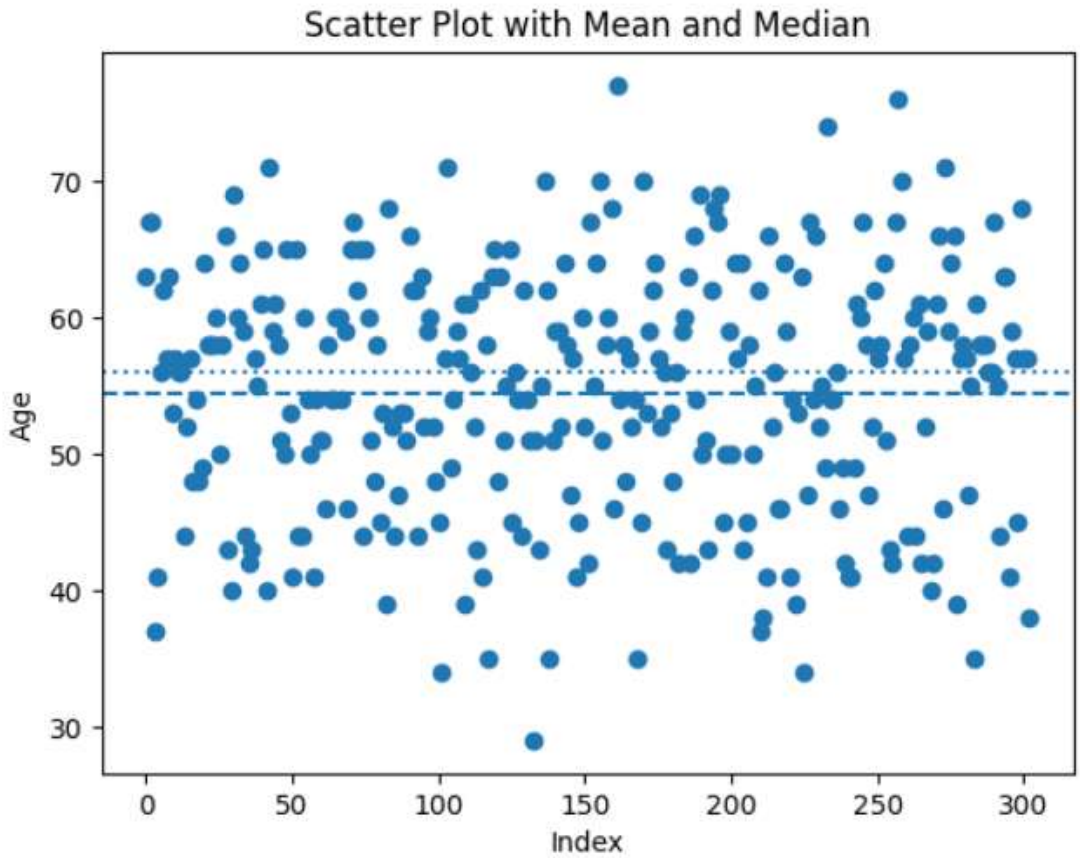
[4]:
```
mean_value = np.mean(column)
median_value = np.median(column)
mode_value = column.mode()[0]

print("Central Tendency Measures:")
print("Mean:", mean_value)
print("Median:", median_value)
print("Mode:", mode_value)
```

```
Central Tendency Measures:
Mean: 54.43894389438944
Median: 56.0
Mode: 58
```

[5]:
```
variance_value = np.var(column)
std_dev_value = np.std(column)
range_value = np.max(column) - np.min(column)

print("\nDispersion Measures:")
print("Variance:", variance_value)
print("Standard Deviation:", std_dev_value)
print("Range:", range_value)
```

```
Dispersion Measures:
Variance: 81.42779030378287
Standard Deviation: 9.02373483119838
Range: 48
```

[6]:
```
plt.figure()
plt.scatter(range(len(column)), column)
plt.axhline(mean_value, linestyle='--')
plt.axhline(median_value, linestyle=':')
plt.title("Scatter Plot with Mean and Median")
plt.xlabel("Index")
plt.ylabel("Age")
plt.show()
```